



**UNIVERSIDAD
SERGIO ARBOLEDA**

Taller 3 corte 2

Diseño e implementación de una gramática LL(1)

Lenguajes de programación

**Yslen Natalia Moreno Gutierrez
Santiago Patiño Sarmiento**

*Universidad Sergio Arboleda
Escuela de ciencias exactas e ingeniería
Bogotá D.C., 30 de septiembre de 2026*

1. Cómo está organizado el programa

Siguiendo la recomendación del profesor, el proyecto se dividió en un archivo por cada parte, y un `main.py` que las coordina:

Archivo	Qué hace
<code>gramatica.py</code>	La gramática y los algoritmos que calculan los conjuntos
<code>lexer.py</code>	Parte léxica: convierte el texto en tokens
<code>parser.py</code>	Parte sintáctica: analizador descendente recursivo y AST
<code>semantica.py</code>	Parte semántica: variables, validaciones y resultados
<code>ast_grafico.py</code>	Dibuja el AST (texto, ventana e imagen)
<code>interactivo.py</code>	Modo interactivo para escribir expresiones
<code>main.py</code>	Une todas las fases

Tabla 1: Archivos del proyecto.

2. La gramática

Primero escribimos la gramática como se escribiría normalmente, pero tenía dos problemas para ser LL(1):

- **Recursión por la izquierda:** una regla como $expr \rightarrow expr + term$ hace que el parser se llame a sí mismo sin avanzar. Se cambió por reglas prima: $expr \rightarrow term\ expr'$ y $expr' \rightarrow + term\ expr' \mid \varepsilon$.
- **Dos opciones que empiezan igual:** la regla $sentencia \rightarrow ID = expr; \mid expr;$ tiene un conflicto, porque una expresión también puede empezar con ID. Se factorizó con una regla nueva, `resto_sent`: después de leer el ID, si viene `=` es una asignación, y si no, es una expresión. Así el parser siempre decide mirando un solo token.

```

GNU nano 8.7.1 gramatica.py
# gramatica.py
# Gramática LL(1) del lenguaje y ALGORITMOS que calculan, a partir de ellas:
# PRIMEROS, SIGUIENTES, PREDICCIÓN, verificación LL(1) y tabla M.
# También sirve para cualquier otra gramática leída desde un archivo .txt:
# eliminación de recursión izquierda, factorización y traza del análisis.

EPS = "ε"
FIN = "$"

# — Gramática del lenguaje —
# Una producción vacía se escribe ["ε"]
GRAMMAR = {
    "programa": [
        ["sentencia", "programa"],
        ["ε"]
    ],
    # Factorización por la izquierda: cada sentencia arranca con un token distinto
    "sentencia": [
        ["ID", "resto_sent", "SEMICOLON"],
        ["NUM", "term", "expr", "SEMICOLON"],
        ["MINUS", "factor", "term", "expr", "SEMICOLON"],
        ["ABS", "LPAREN", "expr", "RPAREN", "term", "expr", "SEMICOLON"],
        ["SIN", "LPAREN", "expr", "RPAREN", "term", "expr", "SEMICOLON"],
        ["COS", "LPAREN", "expr", "RPAREN", "term", "expr", "SEMICOLON"],
        ["TAN", "LPAREN", "expr", "RPAREN", "term", "expr", "SEMICOLON"],
        ["LPAREN", "expr", "RPAREN", "term", "expr", "SEMICOLON"],
    ],
    # Con ASIG es asignación; si no, es una expresión que empezó con ID
    "resto_sent": [
        ["ASIG", "expr"],
        ["term", "expr"]
    ],
    "expr": [
        ["term", "expr"]
    ],
    "expr1": [
        ["PLUS", "term", "expr1"],
        ["MINUS", "term", "expr1"],
        ["ε"]
    ],
    "term": [
        ["factor", "term"]
    ],
    "term1": [
        ["MULT", "factor", "term1"],
        ["DIV", "factor", "term1"],
        ["MOD", "factor", "term1"],
        ["ε"]
    ]
}

363 líneas leídas
Ayuda Guardar Buscar Cortar Ejecutar Ubicación
Salir Leer fich. Reemplazar Pegar Justificar Ir a línea

```

Figura 1: Gramática del lenguaje en gramatica.py.

3. Parte léxica

El analizador léxico recorre el texto con expresiones regulares y lo divide en tokens: números, variables, palabras reservadas (`abs`, `sin`, `cos`, `tan`) y símbolos. Las palabras reservadas se revisan antes que las variables, para que `sin` no se tome como el nombre de una variable. Si encuentra un carácter que no pertenece al lenguaje, muestra un error léxico con la línea.

```

GNU nano 8.7.1 lexer.py
lexer.py
# Analizador léxico para el lenguaje de expresiones matemáticas
# Produce una lista de tokens (tipo, valor, línea) a partir del código fuente

import re

# Orden importa: palabras reservadas ANTES que ID
TOKEN_SPEC = [
    ("NUM", r'\d+(\.\d+)?'),
    ("ABS", r'(?i:\babs\b)'),
    ("SIN", r'(?i:\bsin\b)'),
    ("COS", r'(?i:\bcos\b)'),
    ("TAN", r'(?i:\btan\b)'),
    ("ID", r'[a-zA-Z_]\w*'),
    ("ASIG", r'='),
    ("PLUS", r'+'),
    ("MINUS", r'-'),
    ("MULT", r'*'),
    ("DIV", r'/'),
    ("MOD", r'%'),
    ("LPAREN", r'\('),
    ("RPAREN", r'\)'),
    ("SEMICOLON", r';'),
    ("NEWLINE", r'\n'),
    ("SKIP", r'[\t]+'),
    ("MISMATCH", r'.'),
]

MASTER_PATTERN = re.compile(
    '|'.join(f'(?P<{name}>{pattern})' for name, pattern in TOKEN_SPEC)
)

class LexerError(Exception):
    pass

class Token:
    def __init__(self, type_, value, line):
        self.type = type_
        self.value = value
        self.line = line

    def __repr__(self):
        return f"Token({self.type}, {self.value!r}, línea={self.line})"

def tokenize(source_code: str) -> list:
    tokens = []
    line_num = 1

```

67 líneas leídas

Ayuda Guardar Buscar Cortar Ejecutar Ubicación
 Salir Leer fich. Reemplazar Pegar Justificar Ir a línea

Figura 2: Definición de los tokens en lexer.py.

```

Los PRED de cada no terminal son disjuntos
RESULTADO: la gramática ES LL(1)

=====
FASE 1 – ANÁLISIS LÉXICO
=====
Token(ID, 'a', linea=1)
Token(ASIG, '=', linea=1)
Token(NUM, '10', linea=1)
Token(SEMICOLON, ';', linea=1)
Token(ID, 'b', linea=2)
Token(ASIG, '=', linea=2)
Token(NUM, '3', linea=2)
Token(SEMICOLON, ';', linea=2)
Token(ID, 'c', linea=3)
Token(ASIG, '=', linea=3)
Token(ID, 'a', linea=3)
Token(PLUS, '+', linea=3)
Token(ID, 'b', linea=3)
Token(MULT, '*', linea=3)
Token(NUM, '2', linea=3)
Token(SEMICOLON, ';', linea=3)
Token(ID, 'd', linea=4)
Token(ASIG, '=', linea=4)
Token(NUM, '10', linea=4)
Token(MINUS, '-', linea=4)
Token(NUM, '3', linea=4)
Token(MINUS, '-', linea=4)
Token(NUM, '2', linea=4)
Token(SEMICOLON, ';', linea=4)
Token(ID, 'e', linea=5)
Token(ASIG, '=', linea=5)
Token(ID, 'a', linea=5)
Token(MOD, '%', linea=5)
Token(ID, 'b', linea=5)
Token(SEMICOLON, ';', linea=5)
Token(ID, 'f', linea=6)
Token(ASIG, '=', linea=6)
Token(ID, 'a', linea=6)
Token(DIV, '/', linea=6)
Token(ID, 'b', linea=6)
Token(SEMICOLON, ';', linea=6)
Token(ID, 'g', linea=7)
Token(ASIG, '=', linea=7)
Token(ABS, 'abs', linea=7)
Token(LPAREN, '(', linea=7)
Token(MINUS, '-', linea=7)
Token(NUM, '5', linea=7)
Token(RPAREN, ')', linea=7)
Token(PLUS, '+', linea=7)
Token(SIN, 'sin', linea=7)
Token(LPAREN, '(', linea=7)
Token(NUM, '98', linea=7)

```

Figura 3: Tokens obtenidos de un programa de prueba.

4. Parte sintáctica

El parser es **descendente recursivo**: hay una función por cada no terminal, y cada función mira el token actual para decidir qué regla usar. La función **consume** hace lo que en clase llamamos *emparejar*: si el token es el esperado avanza, y si no, muestra un error sintáctico. Mientras analiza, el parser construye el árbol de sintaxis abstracta (AST).

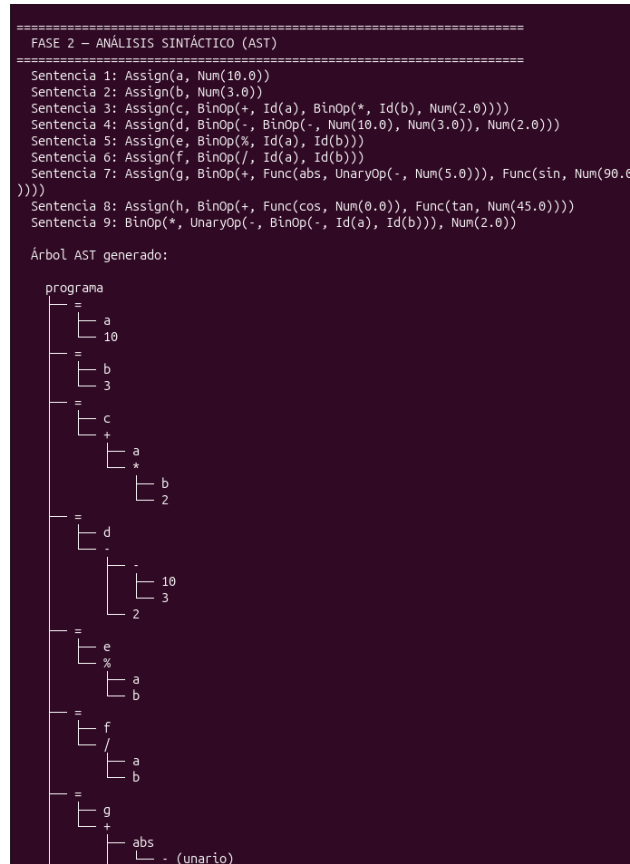


Figura 5: AST en texto

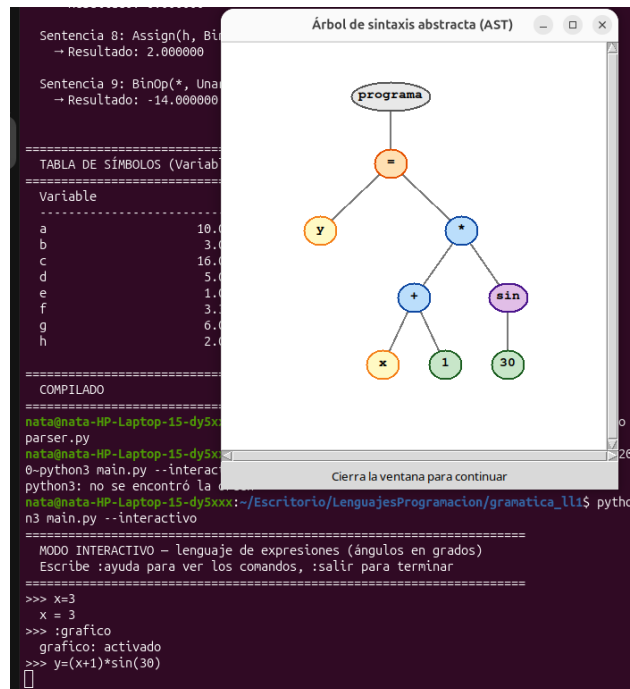


Figura 6: AST grafico

5. Parte semántica

El analizador semántico recorre el AST, calcula el resultado de cada sentencia y guarda las variables en una tabla de símbolos. También revisa los errores que no se pueden ver solo con la gramática: usar una variable que no existe, dividir o sacar módulo entre cero, y calcular la tangente de un ángulo donde no está definida. Las funciones trigonométricas trabajan en grados.



```
GNU nano 8.7.1          semantica.py
# semantica.py
# Analizador semántico y evaluador del AST
# Detecta: variables no declaradas, división por cero
# Evalúa: el resultado numérico de cada sentencia

import math
from parser import (
    NumNode, IdNode, BinOpNode, UnaryOpNode,
    FuncNode, AssignNode, ProgramNode
)

class SemanticError(Exception):
    pass

class SemanticAnalyzer:
    def __init__(self):
        # Tabla de símbolos: nombre → valor numérico
        self.symbol_table = dict = {}

    def analyze(self, node):
        """Punto de entrada: recibe el nodo raíz del AST."""
        if isinstance(node, ProgramNode):
            return self._visit_program(node)
        raise SemanticError(f"[SEMANTICO] Nodo raíz no reconocido")

    def _visit_program(self, node: ProgramNode):
        results = []
        for stmt in node.statements:
            result = self._visit(stmt)
            results.append(result)
        return results

    def _visit(self, node):
        if isinstance(node, NumNode):
            return node.value

        elif isinstance(node, IdNode):
            if node.name not in self.symbol_table:
                raise SemanticError(
                    f"[SEMANTICO] Variable '{node.name}' usada antes de ser declarada"
                )
            return self.symbol_table[node.name]

        elif isinstance(node, AssignNode):
            value = self._visit(node.expr)
            self.symbol_table[node.name] = value
            return value

94 líneas leídas
Ayuda  Guardar  Buscar  Cortar  Ejecutar  Ubicación
Salir  Leer fich.  Reemplazar  Pegar  Justificar  Ir a línea
```

Figura 7: Validaciones semánticas

```

└─ 2
=====
FASE 3 – ANÁLISIS SEMÁNTICO Y EVALUACIÓN
=====
Sentencia 1: Assign(a, Num(10.0))
→ Resultado: 10.000000

Sentencia 2: Assign(b, Num(3.0))
→ Resultado: 3.000000

Sentencia 3: Assign(c, BinOp(+, Id(a), BinOp(*, Id(b), Num(2.0))))
→ Resultado: 16.000000

Sentencia 4: Assign(d, BinOp(-, BinOp(-, Num(10.0), Num(3.0)), Num(2.0)))
→ Resultado: 5.000000

Sentencia 5: Assign(e, BinOp(%, Id(a), Id(b)))
→ Resultado: 1.000000

Sentencia 6: Assign(f, BinOp(/, Id(a), Id(b)))
→ Resultado: 3.333333

Sentencia 7: Assign(g, BinOp(+, Func(abs, UnaryOp(-, Num(5.0))), Func(sin, Num(90.6
))))
→ Resultado: 6.000000

Sentencia 8: Assign(h, BinOp(+, Func(cos, Num(0.0)), Func(tan, Num(45.0))))
→ Resultado: 2.000000

Sentencia 9: BinOp(*, UnaryOp(-, BinOp(-, Id(a), Id(b))), Num(2.0))
→ Resultado: -14.000000

=====
TABLA DE SÍMBOLOS (Variables declaradas)
=====
Variable      Valor
-----
a              10.000000
b              3.000000
c              16.000000
d              5.000000
e              1.000000
f              3.333333
g              6.000000
h              2.000000

=====
COMPILADO
=====
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/gramatica_ll1$ nano
parser.py

```

Figura 8: resultados y tabla de simbolos

6. Conjuntos PRIMEROS, SIGUIENTES y PREDICCIÓN

Los conjuntos no están escritos a mano: el programa los **calcula** a partir de la gramática. Para PRIMEROS y SIGUIENTES se recorren las producciones una y otra vez, agregando símbolos, hasta que ningún conjunto cambia. Con eso se calcula la PREDICCIÓN de cada producción:

$$\text{PRED}(A \rightarrow \alpha) = \begin{cases} \text{PRIMEROS}(\alpha) & \text{si } \alpha \text{ no puede ser vacía} \\ \text{PRIMEROS}(\alpha) - \{\varepsilon\} \cup \text{SIGUIENTES}(A) & \text{si } \alpha \text{ puede ser vacía} \end{cases}$$

Por último, el programa revisa que las predicciones de un mismo no terminal no compartan ningún token. Si no comparten, la gramática es LL(1).

```

GNU nano 8.7.1          gramatica.py

        break
    if hubo_cambio:
        break
    return g, cambios

# --- PRIMEROS ---
def primeros_cadena(g, primeros, alfa):
    resultado = set()
    for x in alfa:
        if es_terminal(g, x):
            resultado.add(x)
            return resultado
        resultado |= primeros[x] - {EPS}
        if EPS not in primeros[x]:
            return resultado
    resultado.add(EPS) # todos pueden ser vacíos (o alfa es ε)
    return resultado

def calcular_primeros(g):
    primeros = {A: set() for A in g}
    cambio = True
    while cambio: # iteración de punto fijo
        cambio = False
        for A, alts in g.items():
            for alfa in alts:
                antes = len(primeros[A])
                primeros[A] |= primeros_cadena(g, primeros, alfa)
                cambios[A] |= len(primeros[A]) != antes
    return primeros

# --- SIGUIENTES ---
def calcular_siguietes(g, inicial, primeros):
    siguientes = {A: set() for A in g}
    siguientes[inicial].add(FIN)
    cambio = True
    while cambio:
        cambio = False
        for A, alts in g.items():
            for alfa in alts:
                for i, B in enumerate(alfa):
                    if es_terminal(g, B):
                        continue
                    antes = len(siguietes[B])
                    beta = primeros_cadena(g, primeros, alfa[i + 1:])
                    siguientes[B] |= beta - {EPS}

Ayuda      Guardar  Buscar  Cortar  Ejecutar  Ubicación
Salir      Leer fich. Reemplazar Pegar  Justificar Ir a línea

```

Figura 9: Algoritmo primeros y siguientes

```

=====
CONJUNTOS PRIMEROS, SIGUIENTES Y PREDICCIÓN (calculados)
=====
PRIMEROS
PRIMEROS(programa ) = { ABS, COS, ID, LPAREN, MINUS, NUM, SIN, TAN, ε }
PRIMEROS(sentencia ) = { ABS, COS, ID, LPAREN, MINUS, NUM, SIN, TAN }
PRIMEROS(resto_sent) = { ASIG, DIV, MINUS, MOD, MULT, PLUS, ε }
PRIMEROS(expr ) = { ABS, COS, ID, LPAREN, MINUS, NUM, SIN, TAN }
PRIMEROS(expr' ) = { MINUS, PLUS, ε }
PRIMEROS(term ) = { ABS, COS, ID, LPAREN, MINUS, NUM, SIN, TAN }
PRIMEROS(term' ) = { DIV, MOD, MULT, ε }
PRIMEROS(factor ) = { ABS, COS, ID, LPAREN, MINUS, NUM, SIN, TAN }

SIGUIENTES
SIGUIENTES(programa ) = { $ }
SIGUIENTES(sentencia ) = { ABS, COS, ID, LPAREN, MINUS, NUM, SIN, TAN, $ }
SIGUIENTES(resto_sent) = { SEMICOLON }
SIGUIENTES(expr ) = { RPAREN, SEMICOLON }
SIGUIENTES(expr' ) = { RPAREN, SEMICOLON }
SIGUIENTES(term ) = { MINUS, PLUS, RPAREN, SEMICOLON }
SIGUIENTES(term' ) = { MINUS, PLUS, RPAREN, SEMICOLON }
SIGUIENTES(factor ) = { DIV, MINUS, MOD, MULT, PLUS, RPAREN, SEMICOLON }

```

Figura 10: calculos

PREDICCIÓN		
1. PRED(programa → sentencia programa	) = { ABS, COS,	
ID, LPAREN, MINUS, NUM, SIN, TAN }		
2. PRED(programa → ε	) = { \$ }	
3. PRED(sentencia → ID resto_sent SEMICOLON	) = { ID }	
4. PRED(sentencia → NUM term' expr' SEMICOLON	) = { NUM }	
5. PRED(sentencia → MINUS factor term' expr' SEMICOLON	) = { MINUS }	
6. PRED(sentencia → ABS LPAREN expr RPAREN term' expr' SEMICOLON	) = { ABS }	
7. PRED(sentencia → SIN LPAREN expr RPAREN term' expr' SEMICOLON	) = { SIN }	
8. PRED(sentencia → COS LPAREN expr RPAREN term' expr' SEMICOLON	) = { COS }	
9. PRED(sentencia → TAN LPAREN expr RPAREN term' expr' SEMICOLON	) = { TAN }	
10. PRED(sentencia → LPAREN expr RPAREN term' expr' SEMICOLON	) = { LPAREN }	
11. PRED(resto_sent → ASIG expr	) = { ASIG }	
12. PRED(resto_sent → term' expr'	) = { DIV, MINUS	
, MOD, MULT, PLUS, SEMICOLON }		
13. PRED(expr → term expr'	) = { ABS, COS,	
ID, LPAREN, MINUS, NUM, SIN, TAN }		
14. PRED(expr' → PLUS term expr'	) = { PLUS }	
15. PRED(expr' → MINUS term expr'	) = { MINUS }	
16. PRED(expr' → ε	) = { RPAREN, SE	
MICOLON }		
17. PRED(term → factor term'	) = { ABS, COS,	
ID, LPAREN, MINUS, NUM, SIN, TAN }		
18. PRED(term' → MULT factor term'	) = { MULT }	
19. PRED(term' → DIV factor term'	) = { DIV }	
20. PRED(term' → MOD factor term'	) = { MOD }	
21. PRED(term' → ε	) = { MINUS, PLU	
S, RPAREN, SEMICOLON }		
22. PRED(factor → LPAREN expr RPAREN	) = { LPAREN }	
23. PRED(factor → MINUS factor	) = { MINUS }	
24. PRED(factor → ABS LPAREN expr RPAREN	) = { ABS }	
25. PRED(factor → SIN LPAREN expr RPAREN	) = { SIN }	
26. PRED(factor → COS LPAREN expr RPAREN	) = { COS }	
27. PRED(factor → TAN LPAREN expr RPAREN	) = { TAN }	
28. PRED(factor → NUM	) = { NUM }	
29. PRED(factor → ID	) = { ID }	
VERIFICACIÓN LL(1)		
Los PRED de cada no terminal son disjuntos		
RESULTADO: la gramática ES LL(1)		

Figura 11: Prediccion y verificacion

7. Gramáticas de las diapositivas

El programa también lee cualquier gramática desde un archivo de texto con el modo **-gramatica**. Con ella hace lo que se vio en clase: elimina la recursión izquierda, factoriza, calcula los conjuntos, verifica si es LL(1), arma la tabla de análisis y muestra la **traza** de una cadena paso a paso.

Diapositiva 6: gramática que no es LL(1)

$A \rightarrow B \text{ uno} \mid \text{dos}$, $B \rightarrow \text{dos}$. Las dos opciones de A pueden empezar con **dos**, así que el programa detecta el conflicto.

```

gramatica.py
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/gramatica_ll1$ python3 main.py --gramatica pruebas/g_diapo06.txt

=====
GRAMÁTICA LEÍDA DE pruebas/g_diapo06.txt
=====
Símbolo inicial: A
A → B uno
A → dos
B → dos

=====
1. ELIMINACIÓN DE RECURSIÓN IZQUIERDA
=====
No hay recursión izquierda inmediata

=====
2. FACTORIZACIÓN POR LA IZQUIERDA
=====
No hay prefijos comunes

=====
3. PRIMEROS, SIGUIENTES Y PREDICCIÓN
=====
PRIMEROS
PRIMEROS(A) = { dos }
PRIMEROS(B) = { dos }

SIGUIENTES
SIGUIENTES(A) = { $ }
SIGUIENTES(B) = { uno }

PREDICCIÓN
1. PRED(A → B uno) = { dos }
2. PRED(A → dos ) = { dos }
3. PRED(B → dos ) = { dos }

VERIFICACIÓN LL(1)
CONFLICTO en A: 'A → B uno' y 'A → dos' comparten { dos }
RESULTADO: la gramática NO es LL(1)

=====
4. TABLA DE ANÁLISIS
=====
TABLA DE ANÁLISIS M[no terminal, token]
M[A, dos] = A → B uno
M[B, dos] = B → dos
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/gramatica_ll1$

```

Figura 12: Diapositiva 6: conflicto LL(1)

Diapositiva 12: factorización

Las dos formas del `if` empiezan igual. El programa saca la parte común y crea I' , igual que en la diapositiva.

```

nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/granatica_ll1$ python3 main.py --granatica pruebas/g_diapo12.txt

=====
GRAMÁTICA LEÍDA DE pruebas/g_diapo12.txt
=====
Símbolo inicial: I
I → if E then I endif
I → if E then I else I endif
I → otra
E → cond

=====
1. ELIMINACIÓN DE RECURSIÓN IZQUIERDA
=====
No hay recursión izquierda inmediata

=====
2. FACTORIZACIÓN POR LA IZQUIERDA
=====
Se factorizó en: I
I → if E then I I'
I → otra
I' → endif
I' → else I endif
E → cond

=====
3. PRIMEROS, SIGUIENTES Y PREDICCIÓN
=====
PRIMEROS
PRIMEROS(I) = { if, otra }
PRIMEROS(I') = { else, endif }
PRIMEROS(E) = { cond }

SIGUIENTES
SIGUIENTES(I) = { else, endif, $ }
SIGUIENTES(I') = { else, endif, $ }
SIGUIENTES(E) = { then }

PREDICCIÓN
1. PRED(I → if E then I I') = { if }
2. PRED(I → otra) = { otra }
3. PRED(I' → endif) = { endif }
4. PRED(I' → else I endif) = { else }
5. PRED(E → cond) = { cond }

VERIFICACIÓN LL(1)
Los PRED de cada no terminal son disjuntos

```

Figura 13: Diapositiva 12: factorización por la izquierda.

8. Pruebas de implementación

Programa válido. Se prueban todas las operaciones, la precedencia ($a + b * 2 = 16$), la asociatividad ($10 - 3 - 2 = 5$), el menos unario y las funciones, también escritas con mayúscula (**Sin**, **Cos**). Sus resultados se ven en la figura de la parte semántica.

Error léxico. El carácter \$ no pertenece al lenguaje.

Error sintáctico. Falta cerrar un paréntesis.

```

=====
FASE 1 - ANÁLISIS LÉXICO
=====
Token(ID, 'a', línea=1)
Token(ASIG, '=', línea=1)
Token(NUM, '5', línea=1)
Token(SEMICOLON, ';', línea=1)
Token(ID, 'b', línea=2)
Token(ASIG, '=', línea=2)
Token(LPAREN, '(', línea=2)
Token(ID, 'a', línea=2)
Token(PLUS, '+', línea=2)
Token(NUM, '2', línea=2)
Token(SEMICOLON, ';', línea=2)

Total de tokens: 11

=====
FASE 2 - ANÁLISIS SINTÁCTICO (AST)
=====

[SINTÁCTICO] Se esperaba 'RPAREN' pero se encontró 'SEMICOLON' (';') en línea 2
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/granatica_ll1$

```

Figura 15: Error sintactico

```

SIGNIFICANTES(factor → ) = { DIV, MINUS, MOD, MUL, PLUS, RPAREN, SEMICOLON }

PREDICCIÓN
1. PRED(programa → sentencia programa ) = { ABS, COS,
ID, LPAREN, MINUS, NUM, SIN, TAN }
2. PRED(programa → ε ) = { $ }
3. PRED(sentencia → ID resto_sent SEMICOLON ) = { ID }
4. PRED(sentencia → NUM term' expr' SEMICOLON ) = { NUM }
5. PRED(sentencia → MINUS factor term' expr' SEMICOLON ) = { MINUS }
6. PRED(sentencia → ABS LPAREN expr RPAREN term' expr' SEMICOLON ) = { ABS }
7. PRED(sentencia → SIN LPAREN expr RPAREN term' expr' SEMICOLON ) = { SIN }
8. PRED(sentencia → COS LPAREN expr RPAREN term' expr' SEMICOLON ) = { COS }
9. PRED(sentencia → TAN LPAREN expr RPAREN term' expr' SEMICOLON ) = { TAN }
10. PRED(sentencia → LPAREN expr RPAREN term' expr' SEMICOLON ) = { LPAREN }
11. PRED(resto_sent → ASIG expr ) = { ASIG }
12. PRED(resto_sent → term' expr' ) = { DIV, MINUS
, MOD, MULT, PLUS, SEMICOLON }
13. PRED(expr → term expr' ) = { ABS, COS,
ID, LPAREN, MINUS, NUM, SIN, TAN }
14. PRED(expr' → PLUS term expr' ) = { PLUS }
15. PRED(expr' → MINUS term expr' ) = { MINUS }
16. PRED(expr' → ε ) = { RPAREN, SE
MICOLON }
17. PRED(term → factor term' ) = { ABS, COS,
ID, LPAREN, MINUS, NUM, SIN, TAN }
18. PRED(term' → MULT factor term' ) = { MULT }
19. PRED(term' → DIV factor term' ) = { DIV }
20. PRED(term' → MOD factor term' ) = { MOD }
21. PRED(term' → ε ) = { MINUS, PLU
S, RPAREN, SEMICOLON }
22. PRED(factor → LPAREN expr RPAREN ) = { LPAREN }
23. PRED(factor → MINUS factor ) = { MINUS }
24. PRED(factor → ABS LPAREN expr RPAREN ) = { ABS }
25. PRED(factor → SIN LPAREN expr RPAREN ) = { SIN }
26. PRED(factor → COS LPAREN expr RPAREN ) = { COS }
27. PRED(factor → TAN LPAREN expr RPAREN ) = { TAN }
28. PRED(factor → NUM ) = { NUM }
29. PRED(factor → ID ) = { ID }

VERIFICACIÓN LL(1)
Los PRED de cada no terminal son disjuntos
RESULTADO: la gramática ES LL(1)

=====
FASE 1 – ANÁLISIS LÉXICO
=====

[LÉXICO] Carácter no reconocido '$' en línea 2
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/gramatica_ll1$

```

Figura 14: Error lexico

Errores semánticos. División por cero, variable no definida y tangente indefinida.

```

[SEMANTICO] se esperaba 'RPAREN' pero se encontró 'SEMICOLON' (;) en línea 2
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/gramatica_ll1$ python3
n3 main.py pruebas/p4_division_cero.txt | tail -1
[SEMANTICO] División por cero
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/gramatica_ll1$ python3
n3 main.py pruebas/p5_variable_no_definida.txt | tail -1
[SEMANTICO] Variable 'x' usada antes de ser declarada
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/gramatica_ll1$ python3
n3 main.py pruebas/p6_tan_indefinida.txt | tail -1
[SEMANTICO] tan(90.0) no está definida (cos = 0)
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/gramatica_ll1$

```

Figura 16: Errores semanticos

9. Modo interactivo

Para probar el lenguaje de forma más cómoda, agregamos un modo interactivo: se escribe una expresión, se presiona Enter y se ve el resultado. Las variables se conservan entre una línea y otra. Tiene comandos para ver los tokens, el árbol, la traza, las variables y los conjuntos.

```
python: no se encontró el orden
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/granatica_ll1$ python3 main.py --interactivo
=====
MODULO INTERACTIVO - lenguaje de expresiones (ángulos en grados)
Escribe :ayuda para ver los comandos, :salir para terminar
=====
>>> x=10
x = 10
>>> y=(x*2)+sin(30)
y = 20.5
>>> :arbol
arbol: activado
>>> :vars
Variable          Valor
-----
x                  10.000000
y                  20.500000
>>>
```

Figura 17: Modo interactivo

El modo interactivo también funciona con las gramáticas de las diapositivas: se escribe una cadena y el programa dice si la acepta o la rechaza. Con estas gramáticas no hay resultados numéricos, porque no tienen parte semántica.

```
nata@nata-HP-Laptop-15-dy5xxx:~/Escritorio/LenguajesProgramacion/granatica_ll1$ python3 main.py --interactivo --gramatica pruebas/g_diapo14.txt
=====
MODULO INTERACTIVO - gramática de pruebas/g_diapo14.txt
=====
A → B uno
A → dos
B → tres
B → cuatro A

La gramática ES LL(1)
Escribe :ayuda para ver los comandos, :salir para terminar
>>> cuatro dos uno
Paso  Pila          Entrada          Acción
1     A $          cuatro dos uno $    A → B uno
2     B uno $      cuatro dos uno $    B → cuatr
o A
3     cuatro A uno $ cuatro dos uno $    consumir
cuatro
4     A uno $      dos uno $          A → dos
5     dos uno $    dos uno $          consumir
dos
6     uno $        uno $              consumir
uno
7     $            $              aceptació
n
→ CADENA ACEPTADA
>>>
```

Figura 18: Modo interactivo con otra gramatica

10. Conclusiones

- Para que una gramática sea LL(1) hay que quitar la recursión por la izquierda y factorizar las opciones que empiezan igual. Así el parser siempre decide con un solo token.
- Calcular los conjuntos con un programa, en lugar de escribirlos a mano, evita errores. En la primera versión, los conjuntos escritos a mano tenían símbolos que no correspondían.
- Separar el programa en léxico, sintáctico y semántico permite saber exactamente en qué parte está cada error.
- El mismo código sirve para cualquier gramática, lo que nos permitió comprobar los ejercicios de las diapositivas y obtener los mismos resultados de clase.